

EXPLORING THE IMPLEMENTATION OF THE LOUVAIN ALGORITHM IN GRAPHBLAS

A Thesis

by

OLUMAYOWA O. OLOWOMEYE

Submitted to the Graduate and Professional School of
Texas A&M University
in partial fulfillment of the requirements for the degree of
MASTER OF SCIENCE

Chair of Committee,	Timothy Davis
Committee Members,	Erick Moreno-Centeno
	Roger Pearce
	Nate Veldt
Head of Department,	Scott Schaefer

May 2026

Major Subject: Computer Science

Copyright 2026 Olumayowa O. Olowomeye

ABSTRACT

Community detection is a fundamental task in graph analytics, with the Louvain algorithm recognized for its efficiency in modularity optimization. However, its inherently sequential design restricts scalability on modern parallel architectures. This thesis seeks to develop a parallel implementation within the GraphBLAS and LAGraph frameworks. Implemented in SuiteSparse:GraphBLAS, Isolate Sets Louvain demonstrates modularity comparable to established frameworks such as NetworkX and iGraph, while achieving significant runtime improvements on large, sparse networks. These results confirm that complex community detection algorithms can be efficiently expressed and parallelized within the algebraic semantics of GraphBLAS, advancing its role as a foundation for scalable and reproducible graph analytics.

DEDICATION

To my family, my mother, father, sister, brother, and extended family, for their steadfast support throughout my master's degree, and to Dr. Timothy Davis and my committee for their guidance and mentorship.

CONTRIBUTORS AND FUNDING SOURCES

Contributors

This work was supported by a thesis committee consisting of Professor Timothy Davis (Advisor) of the Department of Computer Science, Professor Erick Moreno-Centeno of the Department of Industrial & Systems Engineering, and Professors Roger Pearce and Nate Veldt of the Department of Computer Science.

All other work conducted for the thesis was completed by the student independently.

Funding Sources

This thesis was conducted independently without financial support from grants or institutions.

TABLE OF CONTENTS

	Page
ABSTRACT	ii
DEDICATION	iii
CONTRIBUTORS AND FUNDING SOURCES	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	vii
LIST OF TABLES.....	viii
1. BACKGROUND	1
1.1 GraphBLAS	1
1.1.1 SuiteSparse:GraphBLAS	1
1.1.2 Notation and Matrix Representation.....	2
1.1.3 Masks	2
1.1.4 Semirings, Monoids, and Binary Operators	2
1.2 Case Study: Breadth-First Search	3
1.2.1 Classical BFS	3
1.2.2 GraphBLAS BFS	4
1.3 LAGraph.....	6
1.4 Summary	6
2. OVERVIEW OF THE LOUVAIN ALGORITHM	7
3. EXPLORING CURRENT IMPLEMENTATIONS.....	10
3.1 Problem Scope	10
3.2 Familiarization	10
3.3 Exploration of the <i>Louvain</i> Algorithm	11
3.4 Isolate Sets Parallel Louvain.....	12
3.5 Isolate-set Partition Algorithm	13
3.6 Isolate-set-based Parallel Louvain (IPLM)	14
3.7 Luby’s Algorithm and Isolate Sets in GraphBLAS.....	14
4. DEVELOPING LOUVAIN IN LAGRAPH.....	18
4.1 Motivation	18

4.2	Index Binary Operators	19
4.3	Design of the Argmax Semiring Operator	20
4.3.1	User-Defined Types and Motivation	20
4.3.2	Building the Index-Aware Operator	20
4.4	Argmax Reduction and Semiring Construction.....	21
4.5	Integration into the Louvain Algorithm	22
4.5.1	Reformulating The Sequential Louvain in GraphBLAS.....	25
4.6	Louvain with Isolate Sets (LouvainIS)	25
4.6.1	Motivation and Concept	25
4.6.2	GraphBLAS Integration.....	26
4.6.3	Algorithm Outline	26
4.7	Discussion	27
5.	EXPERIMENTAL SETUP AND BASELINE COMPARISONS.....	29
5.1	Baseline Algorithms	29
5.2	Hardware Environment	30
5.3	Datasets	31
5.4	Experimental Methodology.....	32
6.	RESULTS AND ANALYSIS	34
6.1	Impact of Isolate Sets on LouvainIS Performance.....	34
6.1.1	Evaluation of Isolate Sets	34
6.1.2	Speedup over Sequential	35
6.2	Correctness of the Algorithm	35
6.2.1	Correctness Results.....	36
6.2.2	Analysis of Results	36
6.3	Performance	38
6.4	Significance and Performance Implications	39
7.	CONCLUSION AND FUTURE WORK	41
7.1	Conclusion.....	41
7.2	Improving Algorithmic Correctness.....	41
7.3	Future Work	42
	APPENDIX A. TABLES	43
	REFERENCES	45

LIST OF FIGURES

FIGURE		Page
1.1	BFS expressed with adjacency matrix multiplication in GraphBLAS. The frontier expands through masked sparse matrix–vector multiplications.	5
2.1	Illustration of the Louvain method, showing node aggregation after modularity optimization.	9
6.1	Time performance for all algorithms on ca-GrQc and 20000node. Tables A.1 and A.2	38
6.2	Time performance for all algorithms on email-Enron, com-Amazon and 50000node. Tables A.3, A.4, and A.5	39

LIST OF TABLES

TABLE		Page
1.1	Common semirings in GraphBLAS and their applications.....	3
5.1	Hardware and Software Environment for Experiments	31
6.1	Number of Isolate Sets vs Graph Size.....	34
6.2	Speedup of LouvainIS over LouvainSeq across all graphs and thread counts	35
6.3	Average and maximum modularity across multiple tests	36
6.4	Average modularity (Q) and number of communities across algorithms and graphs..	37
A.1	Execution Time (s) for <code>ca-GrQc.mtx</code> Across Algorithms and Threads	43
A.2	Execution Time (s) for <code>20000node.mtx</code> Across Algorithms and Threads.....	43
A.3	Execution Time (s) for <code>email-Enron.mtx</code> Across Algorithms and Threads	43
A.4	Execution Time (s) for <code>50000node.mtx</code> Across Algorithms and Threads.....	44
A.5	Execution Time (s) for <code>com-Amazon.mtx</code> Across Algorithms and Threads	44

1. BACKGROUND

1.1 GraphBLAS

Graphs play a central role in modern applications ranging from social networks and recommender systems to computational biology and cybersecurity. As these problems scale in size and complexity, traditional adjacency list or traversal-based methods often become inefficient, particularly on parallel hardware. The **GraphBLAS** specification addresses this challenge by casting graph problems into the language of sparse linear algebra [1]. In this framework, a graph is represented by its adjacency matrix. Operations such as traversals, connectivity queries, and optimization routines are written as combinations of matrix and vector multiplications. This abstraction allows algorithms to leverage decades of progress in sparse linear algebra, including hardware acceleration and distributed-memory optimizations.

The foundation of GraphBLAS is a compact set of primitives such as sparse matrix–matrix multiplication, sparse matrix–vector multiplication, reductions, and element-wise operations. These primitives are generalized through the concept of *semirings*, which define how scalar multiplication and addition are performed. By choosing different semirings, the same matrix multiplication can express different graph computations. For instance, breadth-first search can be expressed using the Boolean semiring, while shortest-path algorithms can be expressed using the min-plus semiring. In this way, GraphBLAS provides a flexible algebraic language that captures a broad class of algorithms within a single unifying framework.

1.1.1 SuiteSparse:GraphBLAS

The most widely used implementation of the GraphBLAS specification is **SuiteSparse:GraphBLAS**, a high-performance C library developed by Davis [2]. As part of the SuiteSparse project, it offers a memory-efficient backend with support for masked operations, advanced semirings, operator overloading, and GPU acceleration [3]. Since its release, SuiteSparse:GraphBLAS has been adopted in both academic and practical systems. Examples include its use in FalkorDB for query execution,

its integration into MATLAB for sparse matrix multiplication beginning in R2021a, and its bindings to the Julia programming language. Its efficiency on large, sparse data makes it suitable for both rapid prototyping and production-grade applications.

1.1.2 Notation and Matrix Representation

Let $G = (V, E)$ be a directed graph with $|V| = n$ vertices. Its adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$ is defined by

$$\mathbf{A}_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

In weighted graphs, $\mathbf{A}$ may store real values from a domain D . Vectors are denoted as $\mathbf{v} \in D^n$, and the operators $\oplus$ and $\otimes$ represent the additive and multiplicative operations of a semiring $(D, \oplus, \otimes)$. Element-wise multiplication is written as $\odot$.

1.1.3 Masks

An important feature of GraphBLAS is the use of **masks**, which control where the results of an operation are written. For example, computing $\mathbf{C} = \mathbf{AB}$ under a mask $\mathbf{M}$ gives

$$\mathbf{C}\langle\mathbf{M}\rangle = \mathbf{AB},$$

meaning that only entries allowed by $\mathbf{M}$ are updated. Masks may be based on the actual values of $\mathbf{M}$ (valued masks) or on its sparsity pattern (structural masks). They can also be complemented, $\neg\mathbf{M}$, to restrict updates to unmarked locations. This mechanism is particularly effective for algorithms such as BFS, where it prevents revisiting nodes that have already been discovered. More generally, masking avoids per-element branching, making it well suited to parallel execution.

1.1.4 Semirings, Monoids, and Binary Operators

The core of GraphBLAS lies in its various algebraic operators. A **binary operator** $\otimes : D \times D \rightarrow D$ defines multiplication between entries. A **monoid** $(\oplus, e)$ is an associative and commutative operator $\oplus$ with an identity element e , and it governs accumulation and reduction.

A **semiring** $(D, \oplus, e, \otimes)$ combines both, requiring that multiplication distributes over addition. Different semirings yield different algorithms: the arithmetic semiring $(+, \times)$ supports PageRank and power iteration; the Boolean semiring $(\vee, \wedge)$ encodes reachability and BFS; and the min-plus semiring $(\min, +)$ expresses shortest-path problems. // SuiteSparse:GraphBLAS uses a loose definition for semirings. Traditionally, a semiring requires the identity of $\oplus$ to be an annihilator for any element in $\otimes$. This is not reinforced in SuiteSparse:GraphBLAS, allowing for a degree of flexibility that allows for more complex and unique calculations to be done.

Table 1.1: Common semirings in GraphBLAS and their applications.

Semiring	$\oplus$ (Add)	$\otimes$ (Mult)	Example Algorithm
Arithmetic $(+, \times)$	$+$, identity 0	$\times$	PageRank, power iteration
Boolean $(\vee, \wedge)$	$\vee$, identity false	$\wedge$	Reachability, BFS
Min-plus $(\min, +)$	$\min$, identity ∞	$+$	Shortest Path

The flexibility of the semiring abstraction allows a wide variety of graph algorithms to be expressed within a single, unified algebraic framework. By redefining the underlying addition and multiplication operators, operations such as traversal, path finding, connectivity, and clustering can all be formulated as combinations of sparse matrix computations. This approach separates algorithmic logic from data structure details, enabling optimizations in parallelism, memory layout, and execution order to be handled transparently by the GraphBLAS backend. As a result, algorithms that traditionally require specialized implementations can instead be derived through appropriate choices of semirings, highlighting the expressive power and generality of the GraphBLAS model.

1.2 Case Study: Breadth-First Search

To illustrate the algebraic formulation, consider **breadth-first search (BFS)**. Given a source vertex s , BFS explores the graph level by level, visiting all neighbors at distance k before those at distance $k + 1$.

1.2.1 Classical BFS

The traditional implementation of BFS uses a queue to store the current frontier of vertices and a set to record visited nodes. Each iteration dequeues a vertex, explores its neighbors, and

enqueues any unvisited vertices. This process guarantees that vertices are discovered in increasing order of distance from the source, producing shortest unweighted path lengths.

Algorithm 1 Breadth-First Search (Classical)

```

1: procedure BFS( $G = (V, E), s$ )
2:   for all  $v \in V$  do
3:      $visited[v] \leftarrow \mathbf{false}$ 
4:      $distance[v] \leftarrow \infty$ 
5:   end for
6:    $visited[s] \leftarrow \mathbf{true}, distance[s] \leftarrow 0$ 
7:    $Q \leftarrow \{s\}$ 
8:   while  $Q \neq \emptyset$  do
9:      $u \leftarrow \text{DEQUEUE}(Q)$ 
10:    for all  $v \in neighbors(u)$  do
11:      if not  $visited[v]$  then
12:         $visited[v] \leftarrow \mathbf{true}$ 
13:         $distance[v] \leftarrow distance[u] + 1$ 
14:         $\text{ENQUEUE}(Q, v)$ 
15:      end if
16:    end for
17:  end while
18: end procedure

```

1.2.2 GraphBLAS BFS

In GraphBLAS, BFS is expressed using sparse matrix–vector multiplication. Let $\mathbf{A}$ be the adjacency matrix, $\mathbf{f}$ the frontier vector, and $\mathbf{v}$ the visited vector. Each iteration is written as

$$\mathbf{f}_{k+1} \langle \neg \mathbf{v} \rangle = \mathbf{A} \mathbf{f}_k,$$

followed by

$$\mathbf{v} \leftarrow \mathbf{v} \vee \mathbf{f}_{k+1}.$$

This formulation removes explicit queue management, instead advancing the frontier through masked multiplications. The GraphBLAS algorithm is summarized in Algorithm 2.

Algorithm 2 Breadth-First Search (GraphBLAS)

```

1: procedure BFS( $\mathbf{A}, s$ )
2:    $\mathbf{f}(s) \leftarrow 1$  ▷ Initialize frontier with source
3:   for  $level \leftarrow 1$  to  $n - 1$  do
4:      $\mathbf{v}(\mathbf{f}) \leftarrow true$  ▷ Mark visited
5:      $\mathbf{f}(\neg \mathbf{v}) \leftarrow \mathbf{A}\mathbf{f}$  ▷ Advance frontier
6:   end for
7: end procedure

```

The computational cost of each iteration is proportional to the number of nonzeros in $\mathbf{A}$, $O(\text{nnz}(\mathbf{A}))$, which makes this approach well suited to large sparse graphs. Figure 1.1 illustrates the process.

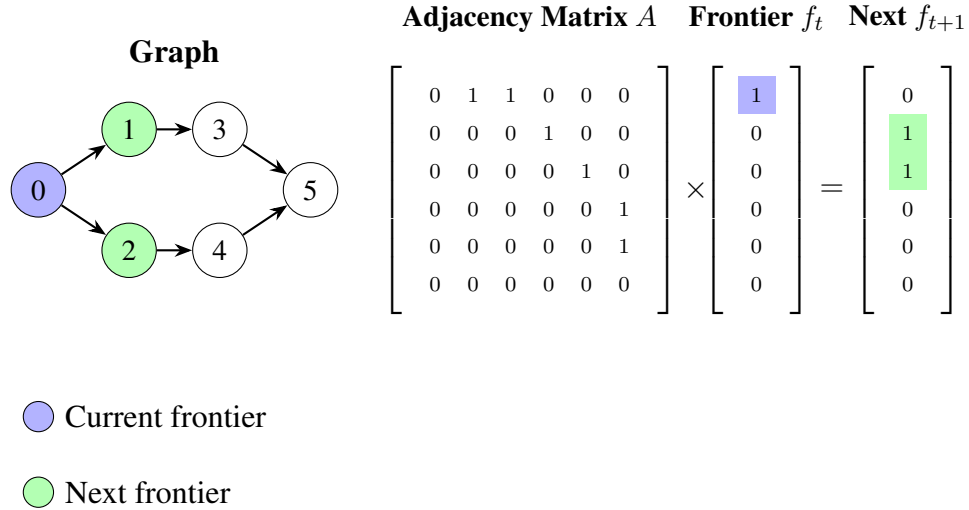


Figure 1.1: BFS expressed with adjacency matrix multiplication in GraphBLAS. The frontier expands through masked sparse matrix–vector multiplications.

1.3 LAGraph

While GraphBLAS provides low-level primitives, **LAGraph** serves as a higher-level algorithmic library that standardizes fundamental algorithms in terms of GraphBLAS operations[4]. It is a community-driven project designed to promote reproducibility and consistency. Its goals include offering a curated collection of reusable algorithms, enforcing consistent input and output formats, and enabling performance comparisons across algorithmic variants.

Currently, LAGraph contains algorithms such as BFS, triangle counting, k -truss, and connected components. However, more advanced routines like Louvain community detection are not yet included, motivating this thesis to investigate their development within the GraphBLAS/LAGraph ecosystem.

1.4 Summary

This chapter introduced the GraphBLAS framework and its algebraic foundation, with SuiteSparse:GraphBLAS serving as a high-performance implementation. Key ideas such as semirings, masks, and monoids were discussed in the context of expressing algorithms. A case study of BFS illustrated how a classical traversal can be reformulated as a sequence of sparse linear algebra operations. Finally, LAGraph was presented as a higher-level library built on GraphBLAS. These foundations set the stage for the design and analysis of community detection algorithms in the following chapters.

2. OVERVIEW OF THE LOUVAIN ALGORITHM

The *Louvain* algorithm is one of the most widely used methods for detecting communities in large graphs. Developed by Blondel et al. [6] at the University of Louvain, it is a greedy optimization approach designed to uncover groups of closely related nodes, or *communities*, by maximizing a quality function known as *modularity*. Introduced and explored by Newman and Girvan [12][11][15], modularity measures how well a partition separates dense clusters of edges within communities from sparser connections between them. Formally, modularity is expressed as

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (2.1)$$

where A_{ij} denotes the adjacency matrix, k_i and k_j are vertex degrees, m is the total edge weight, and $\delta(c_i, c_j)$ indicates whether vertices i and j belong to the same community. A high value of Q corresponds to a division of the graph where most edges fall inside communities rather than between them.

Although modularity is intuitive and widely adopted, its optimization is computationally difficult. Brandes et al. [13] showed that maximizing modularity is NP-hard by proving that the decision version of the problem—determining whether a graph has a partition with modularity above a threshold Q_0 —is NP-complete. This result implies that no polynomial-time algorithm exists for exact modularity maximization in general, unless $P = NP$. In practice, the search space of possible partitions is exponentially large, and modularity landscapes tend to contain many near-optimal solutions with similar values but different structures. For this reason, algorithms such as Louvain are heuristic in nature, aiming to find good approximations efficiently rather than exact optima.

The Louvain algorithm proceeds through two repeating phases. In the first, each vertex is initially placed in its own community. The algorithm then calculates, for each vertex, whether moving it into the community of a neighbor would increase modularity. If so, that vertex is moved

to that community. This process continues until no further local improvements are possible. In the second phase, the communities discovered in the first phase are collapsed into single nodes, producing a smaller aggregated graph in which each node represents a community. The two phases are then repeated on the reduced graph until modularity can no longer be improved. This iterative compression is what allows Louvain to scale to very large networks.

Algorithm 3 Sequential *Louvain* Algorithm

```

1: Input: Graph  $G = (V, E)$ 
2: Output: Community assignment function  $c : V \rightarrow \mathbb{N}$ 
3: Initialize  $c(v) \leftarrow v$  for all  $v \in V$ 
4: Compute total edge weight  $m \leftarrow \sum_{(i,j) \in E} w_{ij}$ 
5: repeat
6:    $Improved \leftarrow \text{false}$ 
7:   for each node  $i \in V$  in random order do
8:     Remove  $i$  from its current community
9:     For each neighboring community  $C_j$ , compute modularity gain  $\Delta Q$ 
10:    Move  $i$  to the community  $C^*$  with maximum positive  $\Delta Q$ 
11:    if  $i$  moved then
12:       $Improved \leftarrow \text{true}$ 
13:    end if
14:  end for
15:  if  $Improved$  then
16:    Aggregate communities to form a new graph  $G'$ , then set  $G \leftarrow G'$ 
17:  end if
18: until  $Improved = \text{false}$ 

```

The strength of Louvain lies in its ability to handle extremely large graphs efficiently, making it a standard tool in network science. Community detection of this kind has practical significance across many domains. In social networks, Louvain can uncover groups of users with shared interests or tightly connected clusters, which may be used for targeted content delivery or misinforma-

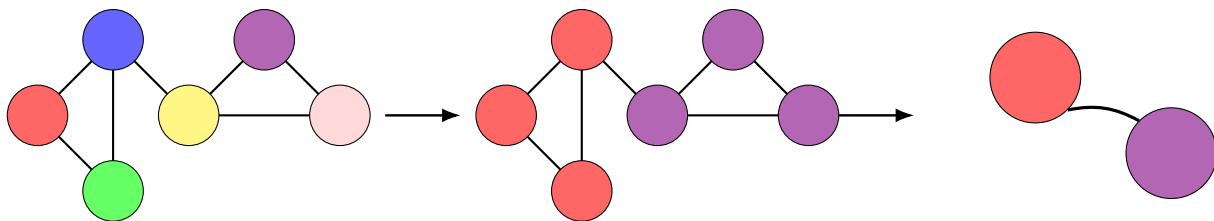


Figure 2.1: Illustration of the Louvain method, showing node aggregation after modularity optimization.

tion detection [18]. In computational biology, it has been applied to the identification of functional modules in genome data [17]. Similarly, in citation networks, it can highlight the evolution of scientific fields and identify emerging research communities.

Despite its popularity, Louvain remains inherently sequential, as the modularity improvement step depends on updating communities one vertex at a time. This sequential nature creates difficulties when attempting to scale the method further on modern parallel architectures. The algorithm becomes more amenable to parallel execution when reformulating Louvain in terms of linear algebraic operations. Such a formulation allows it to exploit the masked sparse matrix operations of SuiteSparse:GraphBLAS, providing both scalability and efficiency. This aligns with LAGraph’s broader mission of developing high-performance, reusable graph algorithms, while also paving the way for parallel community detection methods that can serve as reference implementations within the GraphBLAS ecosystem.

In this thesis, we focus specifically on implementing Phase 1 of the Louvain algorithm—the modularity optimization phase. This phase is the computational core of the method, where vertices are iteratively reassigned to neighboring communities to maximize modularity gain. By concentrating on this stage, we can better evaluate the feasibility and performance of expressing Louvain’s core computations in GraphBLAS primitives without the added complexity of community aggregation in later phases. This approach enables a clearer analysis of parallel scalability, correctness, and performance trade-offs, laying the groundwork for future extensions that incorporate the full multi-phase Louvain procedure.

3. EXPLORING CURRENT IMPLEMENTATIONS

3.1 Problem Scope

This thesis formulates the community detection problem on graphs represented by unweighted adjacency matrices. Let $G = (V, E)$ be an undirected, simple graph with $|V| = n$ vertices and $|E| = m$ edges. Its structure is encoded by the adjacency matrix $A \in \{0, 1\}^{n \times n}$:

$$A_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Because the graph is undirected, A is symmetric. In this formulation, edges are strictly binary, they either exist or they do not. Due to this, edge weights are not considered.

Restricting the scope to unweighted adjacency matrices simplifies the modularity function, which in the weighted case requires degree computations based on edge weights. It also reduces storage costs and allows for the use of highly optimized sparse matrix formats, which is critical for scalability in the GraphBLAS setting. Most importantly, it provides a controlled environment in which to evaluate parallelization strategies for the Louvain algorithm without the added complexity introduced by edge weights.

The remainder of this thesis therefore addresses the problem of detecting communities in large, sparse, unweighted graphs through the implementation and evaluation of a parallel Louvain variant within the SuiteSparse:GraphBLAS and LAGraph ecosystem. We focus on the first phase of Louvain to avoid added complexity. Extensions to weighted graphs and node aggregation are acknowledged as important future directions but remain outside the present scope.

3.2 Familiarization

The early stages of this research focused on developing familiarity with the SuiteSparse:GraphBLAS library and the surrounding LAGraph ecosystem. Resources such as *Graph Algorithms in the*

Language of Linear Algebra [5] were studied alongside implementations already available in LA-Graph. To reinforce this understanding, several small-scale algorithms were developed, including a translation of the `argminmax` function from MATLAB into GraphBLAS.

This phase also included implementing a matrix-based formulation of modularity [17]. In this formulation, modularity Q can be written as

$$Q = \frac{1}{4m} \sum_{v,w} \sum_r \left[A_{vr} - \frac{k_v k_w}{2m} \right] S_{vr} S_{wr} = \frac{1}{4m} \text{Tr}(\mathbf{S}^T \mathbf{B} \mathbf{S}),$$

where A is the adjacency matrix, k_v and k_w are the degrees of nodes v and w , $m = \frac{1}{2} \sum_{i,j} A_{ij}$ is the number of edges, $\mathbf{B} = \mathbf{A} - \frac{\mathbf{k}\mathbf{k}^T}{2m}$ is the modularity matrix, and $\mathbf{S}$ is the binary community assignment matrix. This formulation established the foundation for expressing modularity-based community detection in linear algebraic terms, preparing the way for a Louvain implementation.

3.3 Exploration of the *Louvain* Algorithm

Subsequent work involved a detailed review of Louvain and its extensions. An important work is the Leiden algorithm [14], which is an extension of the Louvain algorithm that introduces a refinement phase to correct one of Louvain’s main shortcomings: fragmented or disconnected communities. After local movement of nodes to optimize modularity, Leiden enforces connectivity by splitting disconnected components and re-optimizing them. Although Leiden represents an important evolution, its implementation lies outside the scope of this thesis.

Parallel versions of Louvain were also studied. The paper by Ghosh et al. [7] presents a distributed-memory parallelization of the Louvain method, a widely-used algorithm for community detection in large graphs. The authors address the scalability limitations of the original Louvain method. Their proposed algorithm is implemented using MPI and demonstrates excellent scalability on large distributed-memory systems. A key contribution of the work is the introduction of novel optimizations to reduce communication overhead and improve load balancing across processes, which are critical challenges in distributed graph processing.

One notable aspect of their implementation is the use of a distributed hash table to manage

community assignments efficiently, allowing multiple processes to work concurrently without extensive synchronization costs. Furthermore, the paper reports strong and weak scaling results on real-world and synthetic graphs, achieving near-linear performance improvement on up to 256 nodes.

The work by Priest, Dunton, and Sanders [8] introduced a novel approach to graph clustering by leveraging *distributed sketches*, a probabilistic data structure technique, to scale clustering algorithms to large-scale graphs. The main motivation behind their approach is to address the computational and memory bottlenecks of traditional clustering algorithms when applied to massive datasets.

The core idea involves representing graph neighborhoods using compact sketches, which significantly reduce memory usage and communication overhead during distributed processing. This sketch-based representation enables efficient similarity computations, which are essential for clustering, without requiring full access to the original graph structure. Of particular relevance is the work of Low et al. [9], who demonstrated a linear algebraic formulation of Louvain in Python using GraphBLAS. Their sequential implementation provided valuable insight into expressing modularity optimization as sparse matrix operations and was reimplemented in LAGraph as a baseline for this research.

Despite these advances, many approaches relied on abstractions outside GraphBLAS’ capabilities. A particularly promising idea comes from *isolate set partitioning*, proposed by Qie et al. [10], in which the graph is partitioned into subsets of vertices that are at least two hops apart. This ensures that nodes can be updated synchronously as a vertex and its neighbour cannot be moved at the same time.

3.4 Isolate Sets Parallel Louvain

Building on this idea, this thesis adopts isolate set partitioning as the central strategy for parallelizing Louvain within GraphBLAS. The method enables multiple nodes to be updated simultaneously, thus allowing the algorithm to be expressed in a linear algebraic formulation.

Some properties of the Isolate Sets are

- One can cover the vertex set V by a collection of disjoint isolate sets (i.e. a partition into isolate sets).
- Within an isolate set, vertices can be updated without interfering with their neighborhoods.

3.5 Isolate-set Partition Algorithm

They propose a graph partition algorithm whose goal is to partition V into disjoint isolate sets $s_1, s_2, \dots, s_k$ such that:

$$\bigcup_i s_i = V, \quad s_i \cap s_j = \emptyset \ (i \neq j).$$

The approach (pseudocode high level) is:

Algorithm 4 Isolate Set Partition

```

1:  $N \leftarrow V$  ▷ Remaining vertices not yet assigned
2: while  $N \neq \emptyset$  do
3:    $N' \leftarrow N$ 
4:   pick an arbitrary seed  $v_j \in N'$ 
5:   initialize a new isolate set  $s_i \leftarrow \{v_j\}$ 
6:   remove from  $N'$  all vertices in the two-hop neighborhood of  $v_j$  (i.e.  $N^+(v_j)$  and neighbors of those)
7:   while  $N' \neq \emptyset$  do
8:     pick a vertex  $u \in N'$ 
9:     add  $u$  to  $s_i$ 
10:    remove from  $N'$  all vertices in  $N^+(u)$ 
11:   end while
12:   remove all vertices in  $s_i$  from  $N$ 
13: end while

```

Key points:

- They use a two-level BFS to identify dependency sets in each step.
- The algorithm ensures that each vertex is assigned once, and no overlaps occur.

- The cost is comparable to BFS and the number of isolates set constructed is small in relative to the total number of nodes.

3.6 Isolate-set-based Parallel Louvain (IPLM)

Given a partition into isolate sets, IPLM proceeds similarly to the conventional Louvain method, but with a controlled pattern of parallelization:

1. **Partitioning:** Precompute isolate sets $s_1, s_2, \dots, s_k$.
2. **Local move (parallel within sets):** For each isolate set s_i in turn, process all vertices in s_i in parallel:
 - Each vertex $v \in s_i$ computes its best modularity gain ΔQ for moving to a neighboring community.
 - Because of the isolate set property, these computations do not interfere with each other (no overlapping dependency sets), so no synchronization delays are needed.
 - After computing, each vertex updates its community assignment (if beneficial), and the updates are consistent (no swaps among vertices in the same s_i).
3. **Aggregation / Coarsening:** After no further modularity increases, aggregate vertices in the same community and rebuild the graph (as in standard Louvain), and repeat.

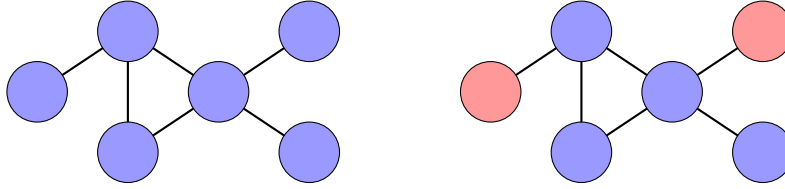
The goal is that due to the Isolate set structure updates to communities can happen at the same time for multiple nodes and latency is avoided.

3.7 Luby's Algorithm and Isolate Sets in GraphBLAS

A connection to Luby's algorithm [16] can be seen from here. Luby introduced a parallel method for computing maximal independent sets (MIS), in which vertices with the highest random priority relative to their neighbors are selected and removed in rounds. With high probability, the algorithm terminates in $O(\log n)$ rounds. Isolate sets can be viewed as a specialization of this approach, but with an additional constraint: selected nodes must be at least two hops apart.

In practice, the construction proceeds by assigning random scores (normalized by degree) to vertices. A vertex is selected if its score is no smaller than those of its immediate neighbors, producing a partial independent set. To enforce the two-hop separation, the squared adjacency matrix A^2 is used: if $(A^2)_{ij} > 0$, then vertices i and j are within two hops and cannot both belong to the isolate set. This filtering ensures that all selected vertices are mutually two hops apart, yielding the desired partition.

Each call to `LAGRAPH_ISOLATESSET` therefore performs a single round of randomized selection followed by the two-hop filtering step using A^2 . Although computing A^2 is expensive, it significantly reduces the active workspace for the rest of Louvain, allowing many vertices to be moved in parallel to their gain communities. A fixed seed is used to make the randomized procedure repeatable for testing.



The algorithmic description is shown in Algorithm 6, and the method can be generalized to n -hop isolate sets by extending the filtering procedure to higher powers of A .

Algorithm 5 Single Isolate Set Extraction

```
1: procedure LAGRAPH_ISOLATESET( $A, ignore, seed$ )  $iset \in \mathbb{B}^n$ 
2:    $degree \leftarrow \sum A$  ▷ out-degree of each node
3:   if  $ignore = \mathbf{NULL}$  then
4:      $candidates \leftarrow \mathbf{true}$ 
5:   else
6:      $candidates \leftarrow \neg ignore$ 
7:   end if
8:    $Seed \leftarrow \mathbf{RANDOMSEED}(seed)$ 
9:    $score \leftarrow Seed \otimes degree$ 
10:   $scoreA \leftarrow score \max .firstA$ 
11:   $neighbor\_max \leftarrow scoreA \max .firstA$ 
12:   $new\_members \leftarrow (score \geq neighbor\_max)$ 
13:   $iset \leftarrow new\_members$ 
14:  return  $iset$ 
15: end procedure
```

Algorithm 6 Isolate Set Partitioning

```
1: procedure LAGRAPH_ISOLATESETS( $A$ ,  $seed$ )
2:   Initialize  $ignore \leftarrow \mathbf{false}$  for all nodes
3:    $result \leftarrow$  empty Boolean matrix
4:   repeat
5:      $iset \leftarrow$  LAGRAPH_ISOLATESET( $A$ ,  $ignore$ ,  $seed$ )
6:     if  $iset$  is empty then
7:       break
8:     end if
9:     Update  $ignore \leftarrow ignore \vee iset$ 
10:    Append  $iset$  to  $result$ 
11:  until  $iset$  is empty
12:  return  $result$ 
13: end procedure
```

4. DEVELOPING LOUVAIN IN LAGRAPH

4.1 Motivation

A central design principle of GraphBLAS is that graph algorithms can be expressed as compositions of sparse linear algebra operations. This abstraction has proven remarkably powerful: breadth-first search, PageRank, and connected components can all be written in terms of semirings. In these cases, the algorithms depend only on numerical values such as edge weights or Boolean indicators of connectivity. The original GraphBLAS specification therefore defined binary operators in a *value-only* manner: operators act only on scalar values and not to the structural indices of those values within a matrix or vector.

In the GraphBLAS standard, a binary operator is formally defined as a mapping

$$\otimes : D \times D \rightarrow D,$$

where D is a scalar domain. The operator takes two inputs $x, y \in D$ and produces a single output $z \in D$. Common examples include arithmetic operations such as addition and multiplication, Boolean operations such as logical AND or OR, and min/max comparisons.

When paired with a monoid, binary operators form the basis of GraphBLAS semirings. This abstraction allows algorithms to be written in a purely algebraic style. For example, breadth-first search is expressed over the `ANY_PAIR` Boolean semiring, single-source shortest paths are written on the `MIN_PLUS` semiring, and PageRank is naturally formulated on the arithmetic semiring. In all of these cases, the operator is “value-only”: the result depends exclusively on the values being combined, and the position of those values in the adjacency structure is irrelevant.

This model is elegant and computationally efficient. Sparse matrix–vector multiplication can be applied uniformly, without special cases or metadata lookups. Yet this abstraction imposes boundaries that become visible when we attempt to encode more complex graph optimization problems.

This design choice offers significant advantages for performance and simplicity, since GraphBLAS kernels can treat all scalars as opaque objects without maintaining index metadata. For many graph primitives, this model is sufficient. Yet certain problems depend critically not only on the numerical value of an edge but also on its position within the graph. Community detection through modularity optimization is one such example: deciding whether a vertex should move to a new community depends not only on the weights of its edges but also on identifiers, degrees, and per-community totals. These cannot be inferred from values alone.

4.2 Index Binary Operators

To bridge this gap, SuiteSparse:GraphBLAS introduces *index-aware binary operators*, implemented as `IndexBinaryOp`. These extend the expressive capacity of GraphBLAS, allowing algorithms to incorporate structural information directly into algebraic operations. The modularity gain ΔQ of moving a vertex i into a community C depends on several quantities: the degree k_i of vertex i , the total degree Σ_{tot}^C of community C , and often a deterministic tie-breaking rule based on vertex identifiers. None of these quantities can be derived from edge weights alone. The adjacency entry A_{ij} tells us only whether or not an edge exists, but not how that edge should be interpreted with respect to global community statistics. Algorithms that require both values and indices cannot be expressed purely in the value-only model. This breaks the uniformity of the model and limits the advantages of optimization and parallelization that GraphBLAS provides. SuiteSparse:GraphBLAS resolves this by allowing binary operators to access index information in addition to values. An index-aware operator has the general form

$$f : (x, i_x, j_x, y, i_y, j_y, \theta) \mapsto z,$$

where x and y are scalar inputs, (i_x, j_x) and (i_y, j_y) are their row and column indices, θ is an optional pointer to user-defined state, and z is the output. This extension unlocks new possibilities. For example, an operator can directly incorporate degree information by using indices to access precomputed degree arrays. It can also enforce deterministic tie-breaking by referencing vertex

IDs. The optional state parameter makes it possible to pass global constants such as the total edge count $2m$ or random seeds for pseudorandomization. By exposing indices, GraphBLAS operators can now encode rules that depend on both the local adjacency pattern and global properties of the graph.

4.3 Design of the Argmax Semiring Operator

A core contribution of this work is the development of an *argmax* semiring that encapsulates the community selection logic within a single algebraic operator. In classical GraphBLAS usage, reduction operators combine scalar values such as numerical weights or Boolean flags. For Louvain, however, each vertex must evaluate a set of candidate communities and select the one that maximizes the modularity gain ΔQ . This requires both the computed gain and the community identifier to be propagated through the reduction. The `argmax_semiring` introduced in this thesis extends GraphBLAS to perform this composite operation efficiently and deterministically.

4.3.1 User-Defined Types and Motivation

The operator construction begins with two user-defined types: a `Theta` struct that stores global state, and an `argmax_tup` struct that stores the local reduction results. The `Theta` record contains a pointer to the degree vector d , the total edge weight m , and a pseudorandom seed used for deterministic tie-breaking. Each `argmax_tup` instance represents a candidate move as a tuple $(\text{score}, \text{tb})$, where `score` encodes the modularity gain, and `tb` is a 64-bit hash derived from the random seed and structural indices. These types are declared as GraphBLAS user-defined types (`GrB_Type`) and passed to subsequent operator constructors.

4.3.2 Building the Index-Aware Operator

The function `make_argmax_tup()` defines the index-aware component of the semiring. It is implemented as a GraphBLAS `IndexBinaryOp`, meaning that it has access to both the scalar values and their corresponding matrix indices during execution. The operator computes the incremental modularity gain for moving vertex i into community C :

$$\Delta Q(i \rightarrow C) = \frac{k_{i \rightarrow C}}{m} - \frac{k_i \Sigma_{\text{tot}}^C}{m^2},$$

where $k_{i \rightarrow C}$ is the total edge weight from vertex i to community C , k_i is the degree of i , Σ_{tot}^C is the sum of degrees in C , and m is half of the total edge weight of the graph.

In Low et al. [9], they acquire this value by using the computation

$$q^S = e_j^S G + \left(-\frac{k_j}{m}\right) k^S$$

to express the modularity contribution of moving vertex j into community T in a purely algebraic form. Here, $e_j^T G$ represents the edge-weight vector between vertex j and the vertices of community s , k_j is the degree of vertex j , m is the total edge weight in the graph, and k^s is the sum of degrees within community s . This formulation allows the modularity update to be implemented as a sequence of sparse matrix-vector operations, enabling a linear-algebraic expression of the Louvain method that can be efficiently executed using GraphBLAS.

The operator packages this result into an `argmax_tup` along with a pseudorandom tie-breaker computed from the vertex indices and seed.

$$\text{tb} = \text{hash}(\text{seed} + i + j).$$

This ensures that ties in ΔQ are resolved deterministically and that the same graph always yields identical results regardless of thread scheduling. During construction, the `Theta` struct is serialized into a `GrB_Scalar` so that each invocation of `make_argmax_tup()` has access to the global constants d , m , and `seed`.

4.4 Argmax Reduction and Semiring Construction

The complementary binary operator `argmax_op()` defines how two tuples are combined during reduction. The rule is straightforward: the tuple with the larger `score` wins, and in the case of equality, the one with the larger tie-breaker value is selected. Formally,

$$\text{argmax_op}(x, y) = \begin{cases} x, & \text{if } x.\text{score} > y.\text{score}, \\ y, & \text{if } x.\text{score} < y.\text{score}, \\ \max(x, y) \text{ by tie-breaker,} & \text{otherwise.} \end{cases}$$

This operator is registered as a GraphBLAS `BinaryOp` over the `argmax_tup` type. Its identity element is the tuple $(-\infty, \text{INT64_MAX}, 0)$ so that it contributes nothing to a reduction. The operator and identity form a user-defined monoid `AM_mon`, which in turn becomes the additive component of a semiring when paired with the index-aware multiplicative operator `MAKEAMTUP_Bop`. The helper function `build_argmax_operator()` encapsulates this process, creating all necessary GraphBLAS objects (`BinaryOp`, `Monoid`, `Semiring`) and returning the final `AM_Semiring` handle. We call this semiring P_{argmax} and denote its use as $\otimes_{P_{\text{argmax}}}$.

4.5 Integration into the Louvain Algorithm

The foundation of this work builds upon the method proposed by Low et al. [9], which first demonstrated that the classical Louvain algorithm could be expressed entirely in terms of sparse linear algebraic operations. Their formulation showed that community detection can be modeled as a sequence of matrix–vector products and reductions, operating on the adjacency matrix A and a community assignment matrix S . This abstraction paved the way for implementations within the GraphBLAS framework and was subsequently adopted into LAGraph as **LouvainSeq**. In this version, modularity optimization proceeds through algebraic transformations: each iteration computes modularity gains using `GrB_vxm()` and updates community assignments via standard arithmetic and Boolean semirings.

However, the original formulation by Low et al [9] relied exclusively on *value-only* binary operators, which operate only on the scalar values contained in matrices and vectors, without reference to their positional indices. To address these limitations, we extend their implementation by introducing the `IndexBinaryOp` mechanism provided in SuiteSparse:GraphBLAS. Specifically, the index-aware operator `make_fp64()` was developed to generate tuples containing the mod-

ularity score and deterministic tiebreaker. It uses index information to retrieve a vertex’s degree and its target community’s total degree from auxiliary arrays stored in user state. This operator is paired with the reduction operator `max_fp64()`, which selects the tuple with the highest modularity gain (and applies the tiebreaker if necessary). Together, these form the *argmax semiring* $\mathcal{S}_{\text{argmax}}$, enabling Louvain’s decision-making step to be written as a purely algebraic reduction over candidate community moves.

The resulting algorithm, shown in Algorithm 7, therefore combines Low et al.’s algebraic structure with the enhanced expressive power of index-aware operators. This integration preserves the elegance and reproducibility of the original method while extending it to support index-dependent computations within the GraphBLAS framework.

Algorithm 7 LouvainSeq: Sequential Linear Algebraic Louvain (GraphBLAS-based)

Require: Adjacency matrix A

Ensure: Community assignment matrix S

```
1: Define custom type tuple_fp64 = { value, tiebreaker }
2: Define index-aware operator make_fp64 () and reduction max_fp64 ()
3: Construct semiring  $\mathcal{S}_{\text{argmax}} = (\text{max\_fp64}, \text{make\_fp64})$ 
4: Compute node degrees  $k \leftarrow \sum_j A(:, j)$ 
5: Compute total edge weight  $m \leftarrow 0.5 \sum_i k_i$ 
6: Initialize  $S \leftarrow I_n$ , and random vector  $y_{\text{rand}}$ 
7: while not converged do
8:   for each vertex  $i$  do
9:      $v \leftarrow A(i, :)$ 
10:     $k_i \leftarrow k[i]$ 
11:     $t_q \leftarrow v_{\text{ANY.PAIR}} S$ 
12:    Temporarily remove  $i$  from its current community
13:     $q_1 \leftarrow (v - \frac{k_i}{m} z) \otimes S$ 
14:     $y_{\text{rand}} \leftarrow y_{\text{rand}} + \text{seed}$ 
15:     $\text{max\_q1} \leftarrow q_1 \otimes_{\mathcal{S}_{\text{argmax}}} y_{\text{rand}}$ 
16:    Extract best community  $k^*$  from  $\text{max\_q1}$ 
17:    Reassign vertex  $i$  to  $k^*$ 
18:   end for
19: end while
20: return  $S$ 
```

While the algorithm remains elegant and concise, it is fundamentally sequential. Each vertex must be evaluated in turn, limiting opportunities for concurrency. This motivates the introduction of the *Isolate Set Louvain* extension presented in the following section, which combines index-aware semantics with graph partitioning to enable efficient parallel modularity optimization.

4.5.1 Reformulating The Sequential Louvain in GraphBLAS

The resulting algorithm, **LouvainSeq**, replaces the arithmetic semiring used by Low et al. with the newly constructed S_{argmax} . Because these computations are independent across isolate sets, they can be executed in parallel with minimal synchronization overhead. The algorithm iterates until no further modularity improvement is detected, at which point the subresults are merged to form the final community assignment matrix S . Every operation candidate generation, reduction, and update is encoded as a standard GraphBLAS call, preserving reproducibility and enabling scalability on modern multicore systems.

4.6 Louvain with Isolate Sets (LouvainIS)

While the P_{argmax} provides an algebraic mechanism for selecting optimal community assignments, the scalability of Louvain is ultimately constrained by its inherently sequential update process. Each vertex’s move can affect the modularity contributions of its neighbors, which creates strong data dependencies and limits parallelism. To address this challenge, this work extends the GraphBLAS-based Louvain formulation with a novel decomposition strategy based on *Isolate Sets*.

4.6.1 Motivation and Concept

An **Isolate Set** is a subset of vertices that can be updated independently during modularity optimization. Vertices within different isolate sets have no immediate interdependencies; that is, the modularity gain of a vertex in one isolate set does not depend on the community assignment of vertices in another. This property enables concurrent updates without violating the correctness of the modularity maximization criterion.

The insight underlying the **LouvainIS** approach is that real-world sparse graphs contain many weakly connected or locally separable structures. By identifying these independent regions of the graph, we can expose fine-grained concurrency and significantly accelerate community detection, especially for large-scale sparse networks.

4.6.2 GraphBLAS Integration

Within the GraphBLAS framework, isolate sets are represented as index vectors that define submatrices of the adjacency matrix A . The overall algorithm proceeds by:

1. Partitioning A into submatrices A_i corresponding to the combination of the adjacency lists of each vector in isolate set i .
2. Applying the argmax semiring operations within each submatrix independently to determine optimal local community moves.
3. Merging the results to update the global community assignment matrix S .

This formulation allows the isolate set computations to be parallelized using standard GraphBLAS kernels such as `GrB_mxv()` and `GrB_reduce()`. Because each submatrix operates on a disjoint subset of vertices, synchronization overhead is minimal, and each step maintains algebraic equivalence with the original Louvain iteration. We extend the previous calculation of:

$$\Delta Q(i \rightarrow C) = \frac{k_{i \rightarrow C}}{m} - \frac{k_i \Sigma_{\text{tot}}^C}{m^2},$$

To

$$\frac{A_{\text{iset}} \cdot S}{m} - \frac{k_{\text{iset}} \cdot (y^T S)}{m^2}$$

This performs the same ΔQ calculation for all nodes in the isolate set using the P_{argmax} with the `argmax_tup` being the component that computes this.

4.6.3 Algorithm Outline

Algorithm 8 summarizes the high-level structure of LouvainIS. Each isolate set is processed using the argmax semiring defined previously, ensuring that modularity updates remain deterministic and reproducible across threads.

This formulation also ensures determinism and reproducibility. Because both the candidate generation and the reduction are encoded in algebraic terms, they can be executed repeatedly with

identical results, independent of thread scheduling.

4.7 Discussion

Index-aware operators extend the expressive reach of GraphBLAS beyond the value-only model, enabling algorithms that depend on both values and structural indices. For modularity-based community detection, they make it possible to encode the Louvain method entirely within the algebraic framework, producing an implementation that is concise, parallelizable, and reproducible.

The implications extend beyond community detection. Many graph algorithms require access to indices or global context: betweenness centrality depends on path enumeration through specific vertices, motif counting requires knowledge of subgraph structure, and label-dependent computations often rely on vertex or edge identifiers. Index-aware operators make it possible to express such algorithms directly in GraphBLAS, broadening the range of problems that can be captured by linear algebraic formulations.

In summary, the development of index-aware binary operators in SuiteSparse:GraphBLAS bridges the gap between pure algebra and graph-theoretic requirements. By making indices first-class participants in operator definitions, GraphBLAS evolves from a framework for value-only computation into a platform capable of supporting richer, more realistic graph analytics.

Algorithm 8 LouvainIS: Louvain with Isolate Sets

Require: Graph G with adjacency matrix A , RNG seed

Ensure: Community assignment matrix S_{result}

```
1: Compute number of nodes  $n \leftarrow$  rows of  $A$ 
2: Initialize  $S \leftarrow I_n$  ▷ Initial community assignment: each node in its own community
3: Compute total edge weight  $m \leftarrow 0.5 \sum_i \text{degree}(i)$ 
4:  $Miset \leftarrow \text{LAGRAPH\_ISOLATESETS}(A, \text{seed})$ 
5:  $isets \leftarrow$  number of Isolate Sets
6:  $k \leftarrow$  degree of each node
7: Allocate array  $A_{iset}[0..]$  to store submatrices for each isolate set
8: for  $i = 0$  to  $isets - 1$  do
9:   Extract nodes in isolate set  $i$  as vector  $iset$ 
10:  Build submatrix  $A_{iset}[i]$  corresponding to  $iset$ 
11: end for
12:  $changed \leftarrow \text{true}$ 
13:  $iter \leftarrow 0$ 
14: while  $changed$  and  $iter < \text{max\_iter}$  do
15:    $changed \leftarrow \text{false}$ 
16:   for  $i = 0$  to  $k - 1$  do
17:     Compute  $y \leftarrow k \cdot S$ 
18:     Compute  $W \leftarrow A_{iset}[i] \cdot S$ 
19:     Update modularity gains  $Wy \leftarrow W \otimes_{P_{argmax}} y$ 
20:     Extract node movements:  $k\_values \leftarrow Wy$ 
21:     Update  $S$  with  $k\_values$ 
22:   end for
23:   Check if  $S$  changed; update  $changed$  accordingly
24: end while
25: return  $S_{result} \leftarrow S$ 
```

5. EXPERIMENTAL SETUP AND BASELINE COMPARISONS

In order to evaluate the performance of the proposed parallel Louvain algorithm, we conducted a series of systematic experiments and compared its behavior against several widely used baselines. The goal of this evaluation is twofold: first, to assess the raw computational efficiency of the new implementation relative to existing approaches; and second, to demonstrate that the algebraic formulation combined with isolate set parallelism provides tangible advantages on large, sparse graphs.

5.1 Baseline Algorithms

Four baselines were selected to cover a broad spectrum of implementation approaches and performance trade-offs.

The first baseline is the **NetworkX Louvain** implementation, written entirely in Python. Although NetworkX is popular in the research community for its ease of use and flexibility, it is not designed for high performance[19]. The Louvain implementation in NetworkX is therefore representative of what many researchers and practitioners encounter when applying community detection in a prototyping or exploratory setting. Its inclusion allows us to highlight the difference between a high-level, interpreted implementation and lower-level, optimized code.

The second baseline is the **iGraph Louvain** implementation, developed in C and exposed to Python via a thin API layer. iGraph provides a well-engineered set of graph analysis algorithms that balance usability and performance[23]. Compared to NetworkX, iGraph Louvain benefits from native C execution and more efficient data structures, resulting in significant speedups despite being *single-threaded*. The Louvain implementation in iGraph does not employ parallelism within its modularity optimization or aggregation phases, although some auxiliary operations in the iGraph framework can use multi-threading when compiled with OpenMP. Including iGraph Louvain therefore provides an important midpoint comparison between a pure Python implementation and explicitly parallel systems, illustrating the gains achievable through low-level optimization

even without thread-level parallelism.

We then compare **GVE-Louvain**, which focuses on efficient parallel execution and scalable memory management. GVE-Louvain represents a state-of-the-art, high-performance baseline for community detection on shared-memory architectures [22]. It leverages fine-grained parallelism and optimized data structures to minimize synchronization overhead and maximize throughput during modularity optimization and graph aggregation. By including GVE-Louvain, we can directly evaluate how the proposed GraphBLAS-based approach compares against an optimized, multi-threaded implementation that was specifically engineered for performance rather than abstraction or portability.

Finally, we evaluate against the **Sequential GraphBLAS Louvain**(LouvainSeq [9] implementation. This algorithm is expressed purely in terms of SuiteSparse:GraphBLAS linear algebraic primitives and serves as the closest point of comparison within the GraphBLAS/LAGraph framework. Crucially, it does not employ isolate set parallelism, so its performance reflects the efficiency of a purely sequential linear algebraic formulation. This makes it a natural baseline against which the parallel approach in this thesis can be measured.

Together, these four baselines span the spectrum from high-level Python to optimized C libraries and parallel high-performance systems, providing a comprehensive context for evaluating the scalability and efficiency of the proposed **LouvainIS** algorithm.

5.2 Hardware Environment

All experiments were executed on a dedicated server to ensure reproducibility and consistency across runs. The machine, `backslash.engr.tamu.edu`, has a dual-socket Intel Xeon system, equipped with E5-2695 v2 processors. Each socket provides twelve physical cores, for a total of twenty-four hardware threads across the system. The base clock frequency is 2.40 GHz, with turbo frequencies reaching 3.20 GHz under load.

The system is non-uniform memory access (NUMA) based, with two NUMA nodes corresponding to the two sockets. This architectural detail is important for parallel performance, since memory locality and thread placement can significantly influence scaling results. To mitigate this

effect, all experiments were pinned to cores in a round-robin fashion across NUMA nodes.

Table 5.1 summarizes the complete hardware and software environment, including compiler versions and library versions. Consistency in the software stack is essential, since even minor differences in compiler optimizations or library internals can skew performance comparisons.

Table 5.1: Hardware and Software Environment for Experiments

Component	Specification
CPU Model	Intel Xeon E5-2695 v2 @ 2.40 GHz
Sockets	2
Cores / Threads	12 cores per socket, 24 total cores / 24 threads
Clock Speed	Base: 2.40 GHz, Max Turbo: 3.20 GHz
Memory	750 GB DDR3
Cache	L1: 768 KiB, L2: 6 MiB, L3: 60 MiB (2 NUMA nodes)
NUMA Nodes	2 (Node 0: CPUs 0–11, Node 1: CPUs 12–23)
Architecture	x86_64 (64-bit, little endian)
Virtualization	Intel VT-x supported
Operating System	Ubuntu 22.04 LTS
Compiler	GCC 11.4.0
Python Version	3.10.12
NetworkX Version	3.4.2
igraph Version	0.11.9
GraphBLAS Version	10.2.0

5.3 Datasets

To ensure that results are representative of real-world conditions, we evaluated the algorithms on a diverse collection of graphs, including both small scientific collaboration networks and very large social and communication networks. The selection spans several orders of magnitude in terms of node and edge counts, thereby testing scalability across different scales.

The **CA-GrQc** dataset represents a scientific collaboration network drawn from arXiv’s General Relativity and Quantum Cosmology category, with just over 5,000 nodes. This dataset exemplifies small but highly clustered social graphs.

The **Email-Enron** dataset consists of an email communication network from the now-defunct Enron corporation. It is significantly larger, with over 36,000 nodes, and serves as a mid-scale real-world benchmark that is sparse yet complex.

Two datasets from the **GraphChallenge 2017 Streaming Partition Challenge** were included: one with 20,000 nodes and another with 50,000 nodes. These synthetic graphs were specifically designed for benchmarking streaming and partitioning algorithms, and their inclusion enables direct comparison with prior results in the literature.

On the large end of the spectrum, we include the **com-Amazon** graph, a product co-purchasing network from Amazon with over 300,000 nodes. These networks test the scalability limits of sequential baselines and highlight the benefits of parallel implementations.

- **CA-GrQc:** 5,242 nodes and 14,496 edges (Source: SNAP [21])
- **Streaming Partition Challenge (20K):** 20,000 nodes and 404,263 edges (Source: GraphChallenge 2017 [20])
- **Email-Enron:** 36,692 nodes and 183,831 edges (Source: SNAP [21])
- **Streaming Partition Challenge (50K):** 50,000 nodes and 1,017,755 edges (Source: GraphChallenge 2017 [20])
- **com-Amazon:** 334,863 nodes and 925,872 edges (Source: SNAP [21])

These datasets were chosen for their variety of sizes, densities, and structural characteristics, providing a diverse basis for evaluating performance and scalability across different graph types.

5.4 Experimental Methodology

The Graphs were stored as .mtx files and tests were set up using the LAGraph library. Each algorithm was executed on the full set of datasets, and all runs were repeated multiple times to account for runtime variability due to caching, thread scheduling, and other system-level effects. Reported timings correspond to the mean over repeated runs, with variance noted when significant.

To ensure fairness, all algorithms were executed in single-threaded mode unless explicit parallelism was inherent to the implementation (as in our proposed approach).

The primary performance metric is wall-clock execution time as well as the modularity and number of communities. We also counted the number of Isolate Sets for the **LouvainIS** to see how it collapsed the numbers of iterations to the smaller isolate set.

In summary, the experimental setup combines a carefully controlled hardware environment, a diverse set of graph datasets, and three complementary baselines. This provides a strong foundation for evaluating the benefits of isolate set parallelism within the GraphBLAS framework.

6. RESULTS AND ANALYSIS

6.1 Impact of Isolate Sets on LouvainIS Performance

In addition to the standard sequential Louvain implementation, our parallel variant, **LouvainIS**, leverages *Isolate Sets* to improve scalability. An isolate set is a subset of vertices that can be updated independently without interfering with other sets. By identifying these groups, the algorithm reduces synchronization overhead and enables concurrent community reassignment within the GraphBLAS runtime. The overall effect is a reduction in convergence time for large, sparse networks where structural independence is common.

6.1.1 Evaluation of Isolate Sets

We evaluated the relationship between graph size and the number of isolate sets identified. For each dataset, we recorded the number of nodes, the average number of isolate sets, and the time required to compute them. Results are shown in Table 6.1.

Table 6.1: Number of Isolate Sets vs Graph Size

Dataset	#Nodes	#Isolate Sets	Computation Time (s)
CA-GrQc	5,242	141	0.80
Streaming Partition Challenge (20K)	20,000	1,482	4.15
Email-Enron	36,692	3,534	12.22
Streaming Partition Challenge (50K)	50,000	1,816	12.05
com-Amazon	334,863	561	15.04

The number of isolate sets increases roughly with graph size but also depends strongly on degree distribution. Dense graphs such as `Email-Enron` exhibit many small sets due to the limited independence of connected vertices, while sparser networks such as `com-Amazon` produce fewer but larger isolate sets. Importantly, isolate set detection remains computationally inexpensive, never exceeding a few seconds even for the largest datasets, making it a practical pre-processing step for parallel execution. Though for this experimental setup we consider the isolate set calculation time as part of the execution time.

6.1.2 Speedup over Sequential

Table 6.2: Speedup of **LouvainIS** over **LouvainSeq** across all graphs and thread counts

Graph / Threads	24	16	8	4	2	1
ca-GrQc.mtx	18.80	16.33	14.77	14.19	13.10	14.26
20000node.mtx	2.60	3.39	2.48	2.13	2.07	2.25
email-Enron.mtx	0.43	0.43	0.42	0.40	0.39	0.46
50000node.mtx	3.12	3.63	3.13	2.52	2.32	2.80
com-Amazon.mtx	41.26	41.68	40.49	79.56	72.83	48.69

The experimental results demonstrate that the **LouvainIS** implementation achieves substantial speedups over the sequential Louvain algorithm across all tested graphs and thread counts. For smaller graphs such as `ca-GrQc.mtx`, the speedup is particularly pronounced, reaching over 14x even with a single thread, indicating the efficiency of the incremental and isolated processing strategies employed by **LouvainIS**. For medium to large graphs, such as `20000node.mtx` and `50000node.mtx`, the speedup remains significant, generally ranging from 2x to 3.6x depending on the thread count, reflecting the benefits of parallelization while also highlighting the overheads introduced by synchronization and load balancing. In extremely large graphs, such as `com-Amazon.mtx`, the speedups vary more dramatically, with some thread configurations achieving over 70x acceleration, suggesting that the parallel **LouvainIS** approach scales very effectively with graph size but may be sensitive to irregularities in graph structure. Overall, these results indicate that **LouvainIS** provides a highly efficient and scalable alternative to the sequential Louvain algorithm, particularly in the context of large, sparse graphs where parallelization can be fully exploited.

6.2 Correctness of the Algorithm

Correctness was evaluated based on two criteria:

1. **Modularity Agreement:** The modularity Q produced by each implementation was compared across datasets to confirm consistent detection of community structure.
2. **Partition Similarity:** Community assignments were examined to ensure that, while numerical labels may differ, the overall community composition remained structurally equivalent.

6.2.1 Correctness Results

Table 6.3: Average and maximum modularity across multiple tests

Graph	Algorithm	Avg Modularity	Max Modularity	Notes
ca-GrQc.mtx	LouvainSeq	0.7775	0.7788	Multi-threaded
	LouvainIS	0.6754	0.6756	Multi-threaded
	NetworkX	0.8623	0.8623	Single-threaded
	iGraph	0.8617	0.8624	Single-threaded
	GVE-Louvain	0.86071	0.86135	Multi-threaded
email-Enron.mtx	LouvainSeq	0.6114	0.6125	Multi-threaded
	LouvainIS	0.4979	0.5145	Multi-thread
	NetworkX	0.6122	0.6122	Single-threaded
	iGraph	0.6129	0.6159	Single-threaded
	GVE-Louvain	0.5993	0.6023	Multi-threaded
20000node.mtx	LouvainSeq	0.7907	0.7907	Multi-threaded
	LouvainIS	0.7907	0.7907	Multi-threaded
	NetworkX	0.7907	0.7907	Single-threaded
	iGraph	0.7907	0.7907	Single-threaded
	GVE-Louvain	0.7907	0.7907	Multi-threaded
50000node.mtx	LouvainSeq	0.8051	0.8051	Multi-threaded
	LouvainIS	0.7957	0.8004	Multi-threaded
	NetworkX	0.8051	0.8051	Single-threaded
	iGraph	0.8051	0.8051	Single-threaded
	GVE-Louvain	0.8051	0.8052	Multi-threaded
com-Amazon.mtx	LouvainSeq	0.8205	0.8131	Multi-threaded
	LouvainIS	0.7957	0.8004	Multi-threaded
	NetworkX	0.9263	0.9263	Single-threaded
	iGraph	0.9251	0.9263	Single-threaded
	GVE-Louvain	0.9262	0.9266	Multi-threaded

6.2.2 Analysis of Results

The results in Tables 6.4 and 6.3 highlight several key findings:

- **LouvainSeq Accuracy:** The sequential GraphBLAS implementation achieves modularity values that closely match both `NetworkX` and `iGraph`. This confirms that the algebraic formulation using index-aware operators faithfully reproduces the community structure of established reference algorithms.
- **LouvainIS Variability:** The parallel variant introduces modest differences in modular-

Table 6.4: Average modularity (Q) and number of communities across algorithms and graphs

Graph	Algorithm	Avg. # Communities
ca-GrQc.mtx	LouvainSeq	779.4
ca-GrQc.mtx	LouvainIS	1541.3
ca-GrQc.mtx	NetworkX	393.0
ca-GrQc.mtx	iGraph	392.8
20000node.mtx	LouvainSeq	32
20000node.mtx	LouvainIS	32
20000node.mtx	NetworkX	32
20000node.mtx	iGraph	32
email-Enron.mtx	LouvainSeq	2116
email-Enron.mtx	LouvainIS	5406.1
email-Enron.mtx	NetworkX	1265.0
email-Enron.mtx	iGraph	1267.0
50000node.mtx	LouvainSeq	44.0
50000node.mtx	LouvainIS	45.4
50000node.mtx	NetworkX	44.0
50000node.mtx	iGraph	44.0
com-Amazon.mtx	LouvainSeq	25095.0
com-Amazon.mtx	LouvainIS	52303.4
com-Amazon.mtx	NetworkX	239.0
com-Amazon.mtx	iGraph	272.0

ity—most noticeable on small or dense graphs such as `ca-GrQc.mtx` and `email-Enron.mtx`.

These deviations arise from concurrent community updates, where parallel threads may select different local optima. Importantly, the overall structure remains consistent, with communities differing only in granularity rather than topology.

- **Community Counts:** **LouvainIS** typically detects a greater number of smaller communities than **LouvainSeq** or reference methods. This most likely is caused by the independence of isolate set processing, which can produce finer-grained partitions when nodes are reassigned concurrently.

Overall, these results demonstrate that the GraphBLAS-based algorithms are *algorithmically sound and reproducible*. **LouvainSeq** confirms the correctness of the algebraic formulation, while **LouvainIS** shows that introducing concurrency through isolate sets preserves the essential struc-

ture of the Louvain method.

A closer examination suggests that the observed variations in modularity for **LouvainIS** may originate from the way isolated sets are defined and applied. Even after vertices are partitioned into isolated sets, modularity can occasionally decrease when updates are performed in parallel. Although the isolated set strategy is intended to prevent direct conflicts—such as adjacent vertices simultaneously moving into each other’s communities—it does not guarantee that every combination of parallel moves will produce a net modularity gain. This behavior reflects an inherent characteristic of the **LouvainIS** algorithm rather than a flaw in its GraphBLAS implementation, highlighting the subtle challenges of maintaining theoretical modularity guarantees under parallel execution.

6.3 Performance

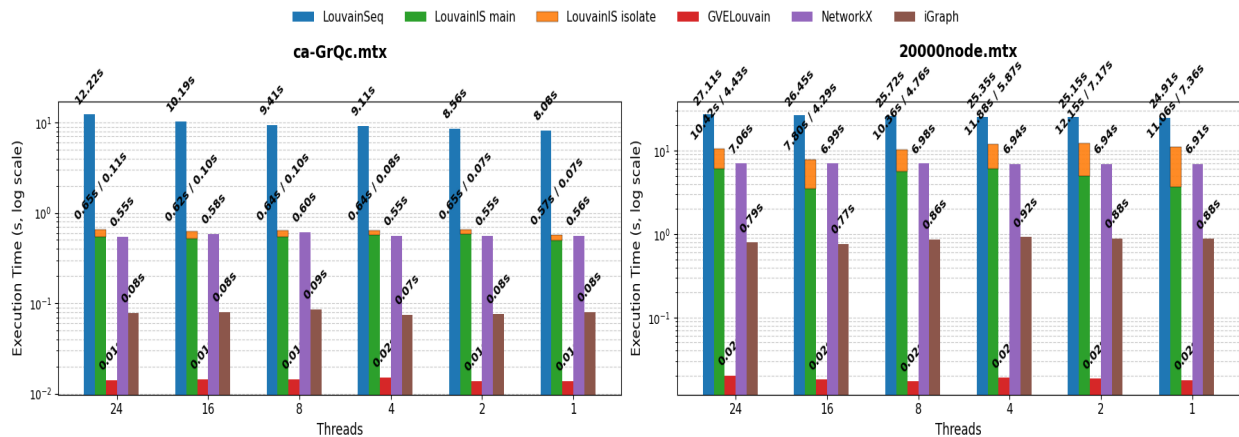


Figure 6.1: Time performance for all algorithms on ca-GrQc and 20000node. Tables A.1 and A.2

From the run times, it can be seen that the Isolate set method is significantly faster than Sequential in GraphBLAS. However, the other algorithms are faster than both with the exception of Networkx Louvain which the Isolate Set runs in similar time to. email-Enron continues to be difficult for **LouvainIS** with it running until the iteration limit. This is again most likely due to its dense nature making it difficult to find a stable modularity with **LouvainIS**.

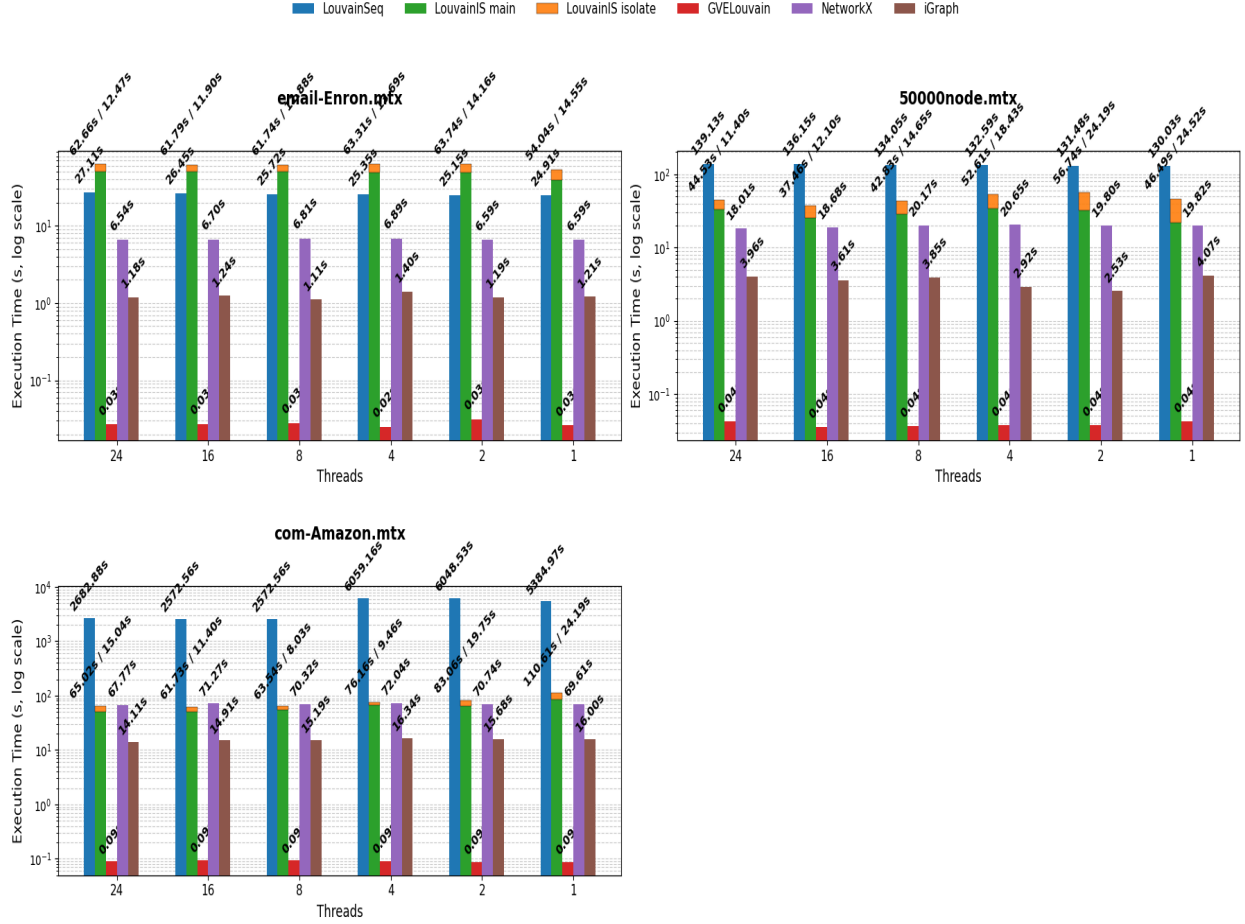


Figure 6.2: Time performance for all algorithms on email-Enron, com-Amazon and 50000node. Tables A.3, A.4, and A.5

6.4 Significance and Performance Implications

The correctness results establish confidence that the GraphBLAS Louvain implementations maintain both theoretical and empirical validity. **LouvainSeq** demonstrates that a modularity-based community detection algorithm can be expressed entirely using index-aware linear algebraic operators without loss of fidelity. **LouvainIS**, while producing slight modularity variations, achieves meaningful parallel speedups by enabling concurrency at the isolate set level.

Performance analysis in subsequent sections confirms that these gains are most pronounced in large, sparse graphs where many vertices belong to disjoint neighborhoods. For dense networks with high interconnectivity, concurrency is naturally limited, and speedups are more modest. Nevertheless, both variants demonstrate stable behavior across all tested datasets, and the algebraic

abstraction incurs minimal overhead relative to procedural implementations.

Collectively, these results validate the feasibility of expressing complex, multi-phase algorithms such as Louvain entirely within the GraphBLAS model. The use of isolate sets provides a scalable parallel extension that maintains determinism, reproducibility, and mathematical transparency. More work needs to be done to further improve the quality of communities produced especially on graphs with more complex structures.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

This thesis presented the design, implementation, and analysis of a parallel Louvain algorithm (phase one) built entirely on the `GraphBLAS` framework and implemented within the `LAGraph` ecosystem. The objective was to express modularity-based community detection as a composition of sparse linear algebra operations, thereby achieving an efficient and scalable formulation that aligns with the goals of the `GraphBLAS` standard.

Two main contributions were achieved. First, the introduction of index-aware operators expanded the expressive capacity of `GraphBLAS`, allowing algorithms to incorporate structural and contextual information directly into algebraic computations. Second, the integration of isolate set parallelism (**LouvainIS**) enabled efficient parallel updates to community assignments by minimizing conflicts between concurrent vertex moves, thereby improving computational throughput while remaining faithful to the modularity optimization framework.

Experimental results confirmed that the purely algebraic formulation (**LouvainSeq**) achieves modularity values that are consistent with established implementations such as `NetworkX`, `iGraph`, and `NetworKit`. The parallel variant (**LouvainIS**) demonstrated significant speedups, especially on large sparse networks, confirming the effectiveness of isolate sets in exposing concurrency. However, as expected, some divergence in modularity scores and community structure was observed, arising from non-deterministic update ordering inherent to parallel execution. These differences fall within the tolerance expected for heuristic optimization algorithms but highlight the ongoing need to balance scalability with correctness.

7.2 Improving Algorithmic Correctness

While the current **LouvainIS** formulation preserves the theoretical principles of modularity maximization, its correctness can be further improved by refining how concurrent updates are handled. Several approaches can help narrow the correctness gap between the sequential and

parallel versions:

- **Isolate Set issues:** While isolate sets minimize direct data conflicts, certain concurrent updates can still reduce overall modularity. Future work should examine heuristics or refinement passes to detect and reverse such cases, restoring the monotonic improvement behavior observed in the sequential algorithm.
- **Iterative Refinement Stage:** Adding a post-processing phase where the final community assignments are re-evaluated sequentially (or partially sequentially) could reconcile any minor inconsistencies introduced during parallel updates. This refinement can restore modularity values closer to those achieved by sequential Louvain. This might be achieved by doing a single iteration going through each Louvain algorithm

Collectively, these strategies point toward a hybrid model that retains the speed of parallel execution while maintaining the correctness and stability of classical Louvain results.

7.3 Future Work

Beyond improving correctness, several directions remain open for future research:

- **Weighted and Directed Graphs:** Extending the formulation to handle edge weights and directionality would generalize the method to a broader class of real-world networks.
- **Dynamic and Streaming Graphs:** Developing incremental or streaming variants of LouvainIS could enable real-time community detection in evolving networks.
- **Full Integration into LAGraph:** Finalizing and contributing a full implementation of Louvain as an official module in the LAGraph repository would provide a standardized, reproducible reference for future research.

In conclusion, this thesis demonstrates that community detection, a challenging algorithm to parallelize can be effectively expressed in the language of sparse linear algebra. By advancing both performance and correctness within the GraphBLAS framework, this work lays the foundation for scalable, transparent, and algebraically grounded community detection in modern graph analytics.

APPENDIX A

TABLES

Table A.1: Execution Time (s) for `ca-GrQc.mtx` Across Algorithms and Threads

Algorithm	24T	16T	8T	4T	2T	1T
LouvainSeq	12.22	10.19	9.41	9.11	8.56	8.08
LouvainIS	0.65	0.62	0.64	0.64	0.65	0.57
<i>(Isolate Time)</i>	(0.11)	(0.10)	(0.10)	(0.08)	(0.07)	(0.07)
GVE-Louvain	0.014	0.014	0.014	0.015	0.014	0.014
NetworkX	0.55	0.58	0.60	0.55	0.55	0.56
iGraph	0.078	0.080	0.085	0.074	0.075	0.080

Table A.2: Execution Time (s) for `20000node.mtx` Across Algorithms and Threads

Algorithm	24T	16T	8T	4T	2T	1T
LouvainSeq	27.11	26.45	25.72	25.35	25.15	24.91
LouvainIS	10.42	7.80	10.36	11.88	12.15	11.06
<i>(Isolate Time)</i>	(4.43)	(4.29)	(4.76)	(5.87)	(7.17)	(7.36)
GVE-Louvain	0.020	0.018	0.017	0.019	0.018	0.018
NetworkX	7.06	6.99	6.98	6.94	6.94	6.91
iGraph	0.79	0.77	0.86	0.92	0.88	0.88

Table A.3: Execution Time (s) for `email-Enron.mtx` Across Algorithms and Threads

Algorithm	24T	16T	8T	4T	2T	1T
LouvainSeq	27.11	26.45	25.72	25.35	25.15	24.91
LouvainIS	62.66	61.79	61.74	63.31	63.74	54.04
<i>(Isolate Time)</i>	(12.47)	(11.90)	(11.88)	(13.69)	(14.16)	(14.55)
GVE-Louvain	0.027	0.027	0.028	0.025	0.031	0.026
NetworkX	6.54	6.70	6.81	6.89	6.59	6.59
iGraph	1.18	1.24	1.11	1.40	1.19	1.21

Table A.4: Execution Time (s) for `50000node.mtx` Across Algorithms and Threads

Algorithm	24T	16T	8T	4T	2T	1T
LouvainSeq	139.13	136.15	134.05	132.59	131.48	130.03
LouvainIS	44.53	37.46	42.83	52.61	56.74	46.49
<i>(Isolate Time)</i>	(11.40)	(12.10)	(14.65)	(18.43)	(24.19)	(24.52)
GVE-Louvain	0.042	0.035	0.036	0.037	0.038	0.043
NetworkX	18.01	18.68	20.17	20.65	19.80	19.82
iGraph	3.96	3.61	3.85	2.92	2.53	4.07

Table A.5: Execution Time (s) for `com-Amazon.mtx` Across Algorithms and Threads

Algorithm	24T	16T	8T	4T	2T	1T
LouvainSeq	2682.88	2572.56	2572.56	6059.16	6048.53	5384.97
LouvainIS	65.02	61.73	63.54	76.16	83.06	110.61
<i>(Isolate Time)</i>	(15.04)	(11.40)	(8.03)	(9.46)	(19.75)	(24.19)
GVE-Louvain	0.089	0.093	0.093	0.089	0.087	0.088
NetworkX	67.77	71.27	70.32	72.04	70.74	69.61
iGraph	14.11	14.91	15.19	16.34	15.68	16.00

REFERENCES

- [1] J. Kepner et al., “Mathematical Foundations of the GraphBLAS,” in 2016 IEEE High Performance Extreme Computing Conference (HPEC), Sept. 2016, pp. 1–9. doi: <https://doi.org/10.1109/HPEC.2016.7761646>.
- [2] T. A. Davis, “Algorithm 1000: SuiteSparse:GraphBLAS: Graph Algorithms in the Language of Sparse Linear Algebra,” *ACM Trans. Math. Softw.*, vol. 45, no. 4, pp. 1–25, Dec. 2019, doi: <https://doi.org/10.1145/3322125>.
- [3] T. A. Davis, “Algorithm 1037: SuiteSparse:GraphBLAS: Parallel Graph Algorithms in the Language of Sparse Linear Algebra,” *ACM Trans. Math. Softw.*, vol. 49, no. 3, pp. 1–30, Sept. 2023, doi: <https://doi.org/10.1145/3577195>.
- [4] T. Mattson et al., “LAGraph: A Community Effort to Collect Graph Algorithms Built on Top of the GraphBLAS,” in 2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), Rio de Janeiro, Brazil: IEEE, May 2019, pp. 276–284. doi: <https://doi.org/10.1109/IPDPSW.2019.00053>.
- [5] J. Kepner and J. Gilbert, “Graph Algorithms in the Language of Linear Algebra”. Society for Industrial and Applied Mathematics, 2011. doi: <https://doi.org/10.1137/1.9780898719918>.
- [6] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *J. Stat. Mech.*, vol. 2008, no. 10, p. P10008, Oct. 2008, doi: <https://doi.org/10.1088/1742-5468/2008/10/P10008>.
- [7] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo[†], Ananth Kalyanaraman, Hao Lu, Daniel Chavarria-Miranda, Arif Khan, and Assefaw H. Gebremedhin, “Distributed Louvain Algorithm for Graph Community Detection,” in 2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS), May 2018, pp. 885–895. doi: [10.1109/IPDPS.2018.00098](https://doi.org/10.1109/IPDPS.2018.00098).

- [8] B. W. Priest, A. Dunton, and G. Sanders, “Scaling Graph Clustering with Distributed Sketches,” in 2020 IEEE High Performance Extreme Computing Conference (HPEC), Sept. 2020, pp. 1–7. doi: <https://doi.org/10.1109/HPEC43674.2020.9286252>.
- [9] T. M. Low, D. G. Spampinato, S. McMillan, and M. Pelletier, “Linear Algebraic Louvain Method in Python,” in 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), New Orleans, LA, USA: IEEE, May 2020, pp. 223–226. doi: <https://doi.org/10.1109/IPDPSW50202.2020.00050>.
- [10] H. Qie, S. Li, Y. Dou, J. Xu, Y. Xiong, and Z. Gao, “Isolate sets partition benefits community detection of parallel Louvain method,” *Sci Rep*, vol. 12, no. 1, p. 8248, May 2022, doi: <https://doi.org/10.1038/s41598-022-11987-y>.
- [11] M. E. J. Newman, “Analysis of weighted networks,” *Phys. Rev. E*, vol. 70, no. 5, p. 056131, Nov. 2004, doi: <https://doi.org/10.1103/PhysRevE.70.056131>.
- [12] M. E. J. Newman and M. Girvan, “Finding and evaluating community structure in networks,” *Phys. Rev. E*, vol. 69, no. 2, p. 026113, Feb. 2004, doi: <https://doi.org/10.1103/PhysRevE.69.026113>.
- [13] U. Brandes et al., “Maximizing Modularity is hard,” Aug. 30, 2006, arXiv: arXiv:physics/0608255. doi: <https://doi.org/10.48550/arXiv.physics/0608255>.
- [14] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: guaranteeing well-connected communities,” *Sci Rep*, vol. 9, no. 1, p. 5233, Mar. 2019, doi: <https://doi.org/10.1038/s41598-019-41695-z>.
- [15] M. E. J. Newman, “Modularity and community structure in networks,” *Proc Natl Acad Sci U S A*, vol. 103, no. 23, pp. 8577–8582, June 2006, doi: <https://doi.org/10.1073/pnas.0601602103>.
- [16] M. Luby. 1986. A simple parallel algorithm for the maximal independent set problem. *SIAM J. Comput.* 15, 4 (1986). doi: <https://doi.org/10.1137/0215074>

- [17] D. Huang, L. Xu, W. Tao, and Y. Li, “Research on Genome Data Recognition and Analysis based on Louvain Algorithm,” July 16, 2024, Computer Science and Mathematics. doi: <https://doi.org/10.20944/preprints202407.1344.v1>.
- [18] P. Held, B. Krause, and R. Kruse, “Dynamic Clustering in Social Networks using Louvain and Infomap Method,” Mar. 08, 2016, arXiv: arXiv:1603.02413. doi: <https://doi.org/10.48550/arXiv.1603.02413>.
- [19] Aric A. Hagberg, Daniel A. Schult and Pieter J. Swart, “Exploring network structure, dynamics, and function using NetworkX”, in Proceedings of the 7th Python in Science Conference (SciPy2008), Gäel Varoquaux, Travis Vaught, and Jarrod Millman (Eds), (Pasadena, CA USA), pp. 11–15, Aug 2008
- [20] E. Kao, V. Gadepally, M. Hurley, M. Jones, J. Kepner, S. Mohindra, P. Monticciolo, A. Reuther, S. Samsi, W. Song, D. Staheli, and S. Smith, “Streaming Graph Challenge: Stochastic Block Partition,” arXiv preprint arXiv:1708.07883, 2017, doi: 10.48550/arXiv.1708.07883.
- [21] J. Leskovec and A. Krevl, “SNAP Datasets: Stanford Large Network Dataset Collection,” Stanford University, 2014. [Online]. Available: <http://snap.stanford.edu/data>
- [22] S. Sahu, “GVE-Louvain: Fast Louvain Algorithm for Community Detection in Shared Memory Setting,” vol. 1189, 2025, pp. 127–139. doi: <https://10.1007/978-3-031-82435-711>.
- [23] louvain-communities: Comparing approaches for community detection using Louvain algorithm,” GitHub, 2025. [Online]. Available: <https://github.com/puzzlef/louvain-communities>.